EXPERIMENT 10

Applying Performance Optimization Techniques: Indexing, Subforms, and Query Tuning

LOW CODE APPLICATION • LABORATORY REPORT

Student Name	Aravindh Shankar
Roll Number	241401012
Programme / Year	B.Tech Computer Science and Business Systems — III Year
Institution	Rajalakshmi Engineering College
Course	Low Code Application

Experiment aim

To understand and apply practical techniques that improve responsiveness and maintainability in a low-code application as record volume grows.

Application / case context

The Gym Enrollment app currently has only a small number of visible records. Performance optimization is therefore primarily a design exercise at this stage. Techniques should be measured on realistic data rather than claiming improvements from a tiny test dataset.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

241401012

Student: Aravindh Shankar • Roll No. 241401012 • Rajalakshmi Engineering College

1. Aim, Objectives and Performance Concepts

Performance optimization aims to reduce response time, unnecessary data retrieval, and maintenance cost without changing business correctness. A small Creator app may feel fast even when its design would struggle at larger scale.

Objectives: identify expensive operations; understand when indexing helps; model repeated child data with subforms or related forms; reduce unnecessary query work; and benchmark before and after changes.

Potential bottlenecks include fetching too many records, repeated queries inside loops, large reports, complex formulas, duplicate workflow execution, and overloading a page with many widgets.

Performance tuning begins with measurement. Choose a repeatable user action, such as opening a filtered report or submitting an enrollment, and record its response time under a known dataset size. Repeat the same operation several times, because one unusually fast or slow run is not a reliable baseline.

Slow performance can originate in different layers. A report may retrieve too many records; a page may render too many elements; a workflow may repeat the same lookup; or a data model may duplicate information unnecessarily. An index cannot fix every bottleneck, so first identify which operation is expensive and then select a technique that addresses it.

Optimization has trade-offs. Normalized relationships can reduce duplicated data but make reporting more complex. Summary fields can make a dashboard faster but require reliable update logic. More complex scripts may reduce repeated work but make the application harder to maintain. Every change should preserve data correctness and be measured against the baseline.

The goal is not to optimize every part of a small app prematurely. Use the benchmark to identify the largest delay, change one component, and remeasure. If the platform manages an optimization internally, record that constraint and focus on the settings and design choices actually available.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

Objectives and scope

This experiment studies evidence-based performance improvement through selective data access, suitable relationships, and repeatable timing tests.

2. Indexing and Search Fields

An index can help a database locate records using selected fields more efficiently, especially for frequent searches or filters. Indexing is not automatically beneficial for every field; it can have storage and write-maintenance costs, and Creator may manage indexing internally or restrict direct control depending on the field and platform.

Prioritize fields frequently used in searches, criteria, and lookups—for example membership plan or start date—only when the platform exposes a supported indexing option and measurement indicates a need.

Do not claim that adding an index improved performance unless before/after timings are measured under comparable conditions. For the small current dataset, report filters are more immediately useful than speculative indexing.

Performance tuning begins with measurement. Choose a repeatable user action, such as opening a filtered report or submitting an enrollment, and record its response time under a known dataset size. Repeat the same operation several times, because one unusually fast or slow run is not a reliable baseline.

Slow performance can originate in different layers. A report may retrieve too many records; a page may render too many elements; a workflow may repeat the same lookup; or a data model may duplicate information unnecessarily. An index cannot fix every bottleneck, so first identify which operation is expensive and then select a technique that addresses it.

Optimization has trade-offs. Normalized relationships can reduce duplicated data but make reporting more complex. Summary fields can make a dashboard faster but require reliable update logic. More complex scripts may reduce repeated work but make the application harder to maintain. Every change should preserve data correctness and be measured against the baseline.

The goal is not to optimize every part of a small app prematurely. Use the benchmark to identify the largest delay, change one component, and remeasure. If the platform manages an optimization internally, record that constraint and focus on the settings and design choices actually available.

Measure at least three runs under the same conditions and compare a representative statistic, such as the median. Record the record count and any filters so that another student can reproduce the test. If timings vary widely, investigate network and rendering effects before attributing the change to the data model.

Do not tune for speed at the expense of correctness. A query that returns fewer records because its criteria are wrong is not an optimization. Compare result counts and sample records before and after every change, especially when reports are used for operational decisions.

Consider the effect of each change on maintenance. A manually maintained summary count can become stale if one workflow path does not update it; a repeated query may be slower but more reliable. The right design is the one that meets the measured need while preserving data integrity.

Use synthetic data for larger benchmarks and keep the test environment separate from actual member data. When comparing results, record the same browser conditions, filter criteria, and dataset size. Do not compare a warm cached run against a cold run and claim the difference came from a code change.

Design / configuration summary

Technique	Use when	Caution
Index/search optimization	A field is repeatedly used to find records and supported indexing is available	Measure first; not every field needs an index
Subform	A parent record contains repeated related entries	Avoid for simple single-value fields
Selective query	Only a subset of records/fields is needed	Ensure criteria do not omit valid records
Workflow conditions	Actions should run only for qualifying records	Test event timing and recursion
Benchmark	Before/after optimization comparison	Use same dataset and workload

3. Subforms and Data Modelling

A subform represents a collection of related rows inside a parent record. It can be appropriate when one enrollment contains repeated child entries, such as emergency contacts, selected training sessions, or family members, and the data must remain linked to the parent.

Do not use a subform simply because a form looks long. A single-valued attribute such as Age or Membership Plan belongs as a normal field. If child rows need independent permissions, reports, or reuse across multiple parents, a separate related form may be more appropriate.

Choose a data model based on relationships and reporting needs. Avoid duplicating the same member details across repeated child rows, because duplicated data becomes inconsistent when a member updates their information.

Performance tuning begins with measurement. Choose a repeatable user action, such as opening a filtered report or submitting an enrollment, and record its response time under a known dataset size. Repeat the same operation several times, because one unusually fast or slow run is not a reliable baseline.

Slow performance can originate in different layers. A report may retrieve too many records; a page may render too many elements; a workflow may repeat the same lookup; or a data model may duplicate information unnecessarily. An index cannot fix every bottleneck, so first identify which operation is expensive and then select a technique that addresses it.

Optimization has trade-offs. Normalized relationships can reduce duplicated data but make reporting more complex. Summary fields can make a dashboard faster but require reliable update logic. More complex scripts may reduce repeated work but make the application harder to maintain. Every change should preserve data correctness and be measured against the baseline.

The goal is not to optimize every part of a small app prematurely. Use the benchmark to identify the largest delay, change one component, and remeasure. If the platform manages an optimization internally, record that constraint and focus on the settings and design choices actually available.

Measure at least three runs under the same conditions and compare a representative statistic, such as the median. Record the record count and any filters so that another student can reproduce the test. If timings vary widely, investigate network and rendering effects before attributing the change to the data model.

Do not tune for speed at the expense of correctness. A query that returns fewer records because its criteria are wrong is not an optimization. Compare result counts and sample records before and after every change, especially when reports are used for operational decisions.

Consider the effect of each change on maintenance. A manually maintained summary count can become stale if one workflow path does not update it; a repeated query may be slower but more reliable. The right design is the one that meets the measured need while preserving data integrity.

Use synthetic data for larger benchmarks and keep the test environment separate from actual member data. When comparing results, record the same browser conditions, filter criteria, and dataset size. Do not compare a warm cached run against a cold run and claim the difference came from a code change.

Procedure and configuration checklist

- Select one measurable operation and establish a baseline.
- Inspect data volume, filters, workflows, and page elements.
- Change one performance factor at a time.
- Repeat measurements and check correctness.

4. Query Tuning and Workflow Efficiency

Fetch only the fields and records required for the task. Apply selective criteria, avoid repeated identical lookups inside loops, and cache or reuse results within the limits of the supported Deluge environment.

Keep workflow conditions narrow so irrelevant records do not trigger expensive actions. Avoid triggering a chain of updates that repeatedly invokes the same workflow unless that behaviour is intentional and tested.

Use reports with suitable filters and sorting. For larger data volumes, test pagination, search behaviour, report load times, and API limits in the actual Creator configuration.

Performance tuning begins with measurement. Choose a repeatable user action, such as opening a filtered report or submitting an enrollment, and record its response time under a known dataset size. Repeat the same operation several times, because one unusually fast or slow run is not a reliable baseline.

Slow performance can originate in different layers. A report may retrieve too many records; a page may render too many elements; a workflow may repeat the same lookup; or a data model may duplicate information unnecessarily. An index cannot fix every bottleneck, so first identify which operation is expensive and then select a technique that addresses it.

Optimization has trade-offs. Normalized relationships can reduce duplicated data but make reporting more complex. Summary fields can make a dashboard faster but require reliable update logic. More complex scripts may reduce repeated work but make the application harder to maintain. Every change should preserve data correctness and be measured against the baseline.

The goal is not to optimize every part of a small app prematurely. Use the benchmark to identify the largest delay, change one component, and remeasure. If the platform manages an optimization internally, record that constraint and focus on the settings and design choices actually available.

Measure at least three runs under the same conditions and compare a representative statistic, such as the median. Record the record count and any filters so that another student can reproduce the test. If timings vary widely, investigate network and rendering effects before attributing the change to the data model.

Do not tune for speed at the expense of correctness. A query that returns fewer records because its criteria are wrong is not an optimization. Compare result counts and sample records before and after every change, especially when reports are used for operational decisions.

Consider the effect of each change on maintenance. A manually maintained summary count can become stale if one workflow path does not update it; a repeated query may be slower but more reliable. The right design is the one that meets the measured need while preserving data integrity.

Use synthetic data for larger benchmarks and keep the test environment separate from actual member data. When comparing results, record the same browser conditions, filter criteria, and dataset size. Do not compare a warm cached run against a cold run and claim the difference came from a code change.

Verification checklist

- Measure the same operation before and after.
- Keep dataset size and conditions consistent.
- Confirm the optimization did not change results.

5. Benchmarking and Conclusion

Create a baseline: record the test dataset size, the operation performed, and the response time over multiple runs. Make one change at a time and repeat the same test. Compare median or average times and note any functional regressions.

Suggested benchmark: open a filtered enrollment report with 50, 500, and 5,000 synthetic records if the environment and plan permit. Do not use real personal or health data for load tests.

Conclusion: optimization should be evidence-driven. Indexing, appropriate subform design, selective queries, and efficient workflows can help at scale, but the correct technique depends on the data model and the actual bottleneck.

Performance tuning begins with measurement. Choose a repeatable user action, such as opening a filtered report or submitting an enrollment, and record its response time under a known dataset size. Repeat the same operation several times, because one unusually fast or slow run is not a reliable baseline.

Slow performance can originate in different layers. A report may retrieve too many records; a page may render too many elements; a workflow may repeat the same lookup; or a data model may duplicate information unnecessarily. An index cannot fix every bottleneck, so first identify which operation is expensive and then select a technique that addresses it.

Optimization has trade-offs. Normalized relationships can reduce duplicated data but make reporting more complex. Summary fields can make a dashboard faster but require reliable update logic. More complex scripts may reduce repeated work but make the application harder to maintain. Every change should preserve data correctness and be measured against the baseline.

The goal is not to optimize every part of a small app prematurely. Use the benchmark to identify the largest delay, change one component, and remeasure. If the platform manages an optimization internally, record that constraint and focus on the settings and design choices actually available.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

24110012

Learning outcomes

- Explained indexing, subforms, and query efficiency.
- Designed repeatable performance measurements.
- Recognized the trade-offs of optimization.

Conclusion

Conclusion: optimization should be evidence-driven. Indexing, appropriate subform design, selective queries, and efficient workflows can help at scale, but the correct technique depends on the data model and the actual bottleneck.

References

Zoho Creator Help: https://www.zoho.com/creator/help/

Zoho Deluge task documentation: https://www.zoho.com/deluge/help/

Zoho Creator performance and limits guidance in the official help centre.

6. Measurement Plan and Optimization Trade-Offs

Choose a measurable target before optimizing, such as report load time, workflow completion time, or the time required to find a record. Measure the same operation under the same data volume and filters before and after a change.

Separate database/query cost from user-interface cost. A slow page may be caused by many embedded elements, a large report, repeated lookups, or a complex formula rather than a missing index. Optimize the actual bottleneck.

Every optimization introduces trade-offs. More complex relationships can improve normalization but make reports harder to configure; additional summaries can improve dashboard speed but require update logic; and extra indexes may not be available for direct configuration in a managed platform.

Measure at least three runs under the same conditions and compare a representative statistic, such as the median. Record the record count and any filters so that another student can reproduce the test. If timings vary widely, investigate network and rendering effects before attributing the change to the data model.

Do not tune for speed at the expense of correctness. A query that returns fewer records because its criteria are wrong is not an optimization. Compare result counts and sample records before and after every change, especially when reports are used for operational decisions.

Practical application to the lab

The Gym Enrollment app currently has only a small number of visible records. Performance optimization is therefore primarily a design exercise at this stage. Techniques should be measured on realistic data rather than claiming improvements from a tiny test dataset.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24114012

7. Optimization Test Plan and Final Recommendations

Use representative synthetic data and repeat each test multiple times. Record the dataset size, filter conditions, browser/device context, and observed response time so the comparison is reproducible.

Do not add complexity just to mention an optimization technique. If the platform manages indexing automatically, document that constraint and focus on query selectivity, report design, workflow efficiency, and correct relationship modelling.

Final recommendation: optimize only after profiling, change one variable at a time, and preserve functional correctness as the primary requirement.

Measure at least three runs under the same conditions and compare a representative statistic, such as the median. Record the record count and any filters so that another student can reproduce the test. If timings vary widely, investigate network and rendering effects before attributing the change to the data model.

Do not tune for speed at the expense of correctness. A query that returns fewer records because its criteria are wrong is not an optimization. Compare result counts and sample records before and after every change, especially when reports are used for operational decisions.

Practical application to the lab

The Gym Enrollment app currently has only a small number of visible records. Performance optimization is therefore primarily a design exercise at this stage. Techniques should be measured on realistic data rather than claiming improvements from a tiny test dataset.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140_012